



Margin-Lens

A Profit-Margin Analysis Application (Vue 3 Frontend)

Project Report

Client-Side Application & API Integration

Student Name:	Umeez Anzar (-087) Talha Shafique (-043)
Course:	Web Engineering
Instructor:	Sir Museb
Date:	16th-07-2026

1. Introduction

MarginLens is a small business tool that answers one question a spreadsheet usually hides: is a product actually profitable once every hidden cost is counted? Most sellers judge a product's margin by subtracting the purchase cost from the selling price. MarginLens calls that number the Perceived Margin. It then asks the seller for the costs that rarely make it into that quick calculation — return rate, payment gateway fees, packaging, shipping and miscellaneous costs — and uses them to compute the Real Margin. The difference between the two is the Margin Gap, the illusion the application is named after.

This deliverable covers the client application: a Vue 3 single-page app that lets a seller manage products and categories, record the hidden-cost profile of a product, and view a report comparing its perceived and real margins. The app is built to talk to a REST API (expected at <http://localhost:3000>) for every piece of data it shows, so the frontend itself keeps no permanent data of its own.

1.1 Technologies used

Tool	Why we used it
Vue 3 (Composition API)	Builds each page as a component using ref/reactive state and <code><script setup></code> .
Vite	Development server and build tool; gives fast reload while working on the views.
Vue Router	Maps a URL such as <code>/product/create</code> to the component that should be shown.
Axios	Sends HTTP requests (GET/POST/PUT/DELETE) from the views to the backend API.
Prettier	Keeps the Vue/JS files formatted consistently across the project.

1.2 Core idea behind the app

Term	Meaning
Perceived Margin	Selling price minus purchase cost only — the margin most sellers assume they are making.
Hidden costs	Return rate %, payment gateway fee %, packaging, shipping and miscellaneous cost per unit.
Real Margin	Perceived margin after the hidden costs are also subtracted.
Margin Gap	How far the perceived margin is from the real margin — the size of the illusion.

2. Project Structure

The client lives in its own folder, WebProject, and is a standard Vue 3 + Vite project. There is no backend code in this folder — the app is written to call an external API and does not bundle a server.



2.1 Why the app is organised this way

File / folder	Its job
main.js	Creates the Vue app, installs the router and mounts it to #app.
App.vue	Renders the top navigation bar (Dashboard / Products / Categories) and the current page.
router/index.js	Declares every route and which view component handles it.
views/	One component per screen. Each view loads its own data with Axios in onMounted().
Validators/Validator.js	Holds the validation rules for a product, a category and a cost profile in one place.
components/icons/LoaderComponent.vue	A reusable loading overlay shown while a view is waiting on the API.

3. Application Architecture

MarginLens is a single-page application: Vue Router swaps the component shown inside `<RouterView>` without a full page reload. There is no central store such as Pinia or Vuex — each view keeps its own state with `ref()` and `reactive()` and fetches what it needs directly from the API when it is opened. This keeps every page simple and self-contained, though it also means, for example, that the product list is reloaded from the server every time the Products page is opened rather than being cached.

3.1 Routing table

Path	View	Purpose
/	Dashboard.vue	Products ranked by margin gap, worst illusion first.
/products	ViewProducts.vue	Full product list with edit, report and delete actions.
/product/create	CreateProduct.vue	Form to add a new product.
/product/edit/:id	EditProduct.vue	Form to update an existing product.
/categories	Categories.vue	Add a category and see all existing categories.
/cost-profile/:id	CostProfile.vue	Enter the hidden costs for a product and trigger the margin calculation.
/product/report/:id	Report.vue	Perceived vs. real margin report for one product.

main.js from Deliverable 1 conventions:

```
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: '/', name: 'dashboard', component: Dashboard },
    { path: '/product/create', name: 'create-product',
      component: () => import('../views/CreateProduct.vue') },
    { path: '/products', name: 'products',
      component: () => import('../views/ViewProducts.vue') },
    { path: '/categories', name: 'categories',
      component: () => import('../views/Categories.vue') },
    { path: '/cost-profile/:id', name: 'cost-profile',
      component: () => import('../views/CostProfile.vue') },
    { path: '/product/report/:id', name: 'report',
      component: () => import('../views/Report.vue') },
    { path: '/product/edit/:id', name: 'edit-product',
      component: () => import('../views/EditProduct.vue') },
  ],
})
```

4. Data Handled by the App

The frontend does not define its own database schema — that lives on the API side — but every form and table in the app agrees on the same shape of data. The fields below are the ones the views actually read and write.

Entity	Fields used by the views
Category	id, name
Product	id, name, categoryId, category (nested, for its name), sellingPrice, purchaseCost, currencyCode
Cost Profile	returnRatePercent, paymentFeePercent, packagingCost, shippingCost, miscCost (submitted against a product id)
Margin Report	productName, perceivedMargin, realMargin, marginGap, totalHiddenCost, purchaseCostPkr, sellingPricePkr, calculatedAt

5. Client-Side Validation

src/Validators/Validator.js centralises the rules that decide whether a form is ready to submit, so the same rule for, say, a positive selling price is not written out separately in every component. Each of the three forms (product, category, cost profile) imports the checks it needs.

```
export function
validateProduct(product) {
  let isValid = false
  if (product.name.trim()
  && product.categoryId >
  0 &&
  product.selling_price > 0
  &&
  product.buying_price > 0
  && product.currencyId >
  0) {
    isValid = true
  }
  return isValid
}

export function
validateCostProfile(cost) {
  let isValid = false
  if (cost.product_id === 0
  || cost.return_rate < 0 ||
  cost.payment_gateway_fee
  < 0
```

```

    || cost.packaging_cost <
0 || cost.shipping_cost < 0
|| cost.miscellaneous < 0) {
    isValid = false
  } else {
    isValid = true
  }
  return isValid
}

```

Function	What it checks
validateProduct	Name is not blank, a category and currency were picked, and both prices are greater than zero.
ValidateCategory	A category name was actually typed before it is submitted.
validateCostProfile	None of the five cost fields (return rate, gateway fee, packaging, shipping, misc) is negative.

Alongside these shared functions, each form also gives immediate feedback field-by-field: an `@input` or `@change` handler updates an errors object as soon as the user leaves a required field empty or picks an invalid option, so the red error text (e.g. "Name is required") appears without waiting for a submit click.

6. Pages and Their Responsibilities

Page	Responsibility
Dashboard	Loads every product's margin report (GET <code>/products/ranked</code>) and lists them from the biggest margin gap down, with a link into that product's full report.
View Products	Lists all products with their category, prices and currency; links out to Edit and Report, and can delete a product.
Create Product	A form for name, category, selling price, purchase cost and currency. On save it sends the product to the API and redirects straight into that product's Cost Profile page.
Edit Product	The same fields as Create Product, pre-filled, for updating a product that already exists.
Categories	A small form to add a category plus a table of every category already created.
Cost Profile	Where the hidden costs are entered for a specific product. Saving this is what triggers the margin calculation on the backend.
Report	Three stat cards (Perceived / Real / Margin Gap) plus a line-by-line cost breakdown for one product.

Every view that talks to the API shows the shared `LoaderComponent` overlay while `isLoading` is true, so the screen never looks frozen while a request is in flight.

7. Communicating with the Backend (Axios)

Unlike a version of this app that centralises requests in one `services/api.js` file, each view here calls axios directly against the API's base address, `http://localhost:3000`. This keeps every view self-contained, at the cost of repeating the base URL in several files.

Method	Endpoint	Called from
GET	/products/ranked	Dashboard.vue
GET	/products	ViewProducts.vue
POST	/products/add	CreateProduct.vue
GET	/product/:id	EditProduct.vue
PUT	/product/update/:id	EditProduct.vue
DELETE	/product/delete/:id	ViewProducts.vue
GET	/categories	CreateProduct.vue, EditProduct.vue, Categories.vue
POST	/categories/add	Categories.vue
POST	/product/cost-profile/add/:id	CostProfile.vue
GET	/product/report/:id	Report.vue

An example, the Cost Profile form submitting and moving on to the report:

```

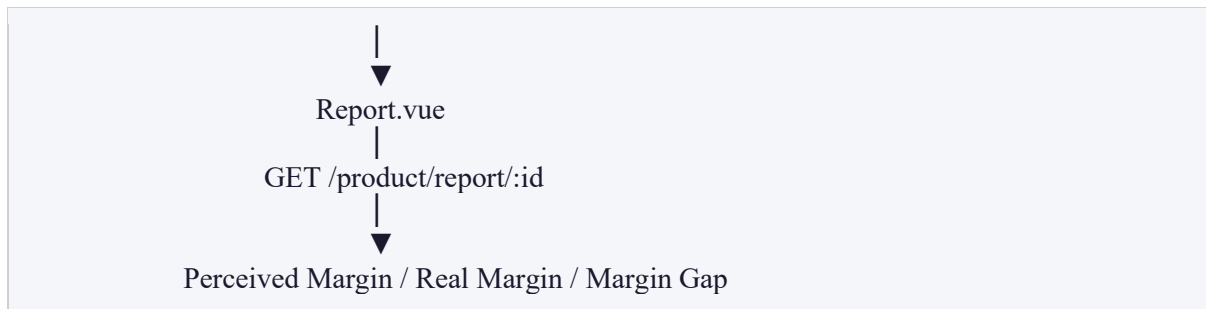
async function saveAndCalculateCost() {
  isLoading.value = true
  const res = await axios.post(
    `http://localhost:3000/product/cost-profile/add/${product_id}`,
    cost
  )
  router.push(`/report/product/${product_id}`)
}

```

8. Application Flow

The diagram below follows the path a seller takes to find out a product's real margin: add the product, describe its hidden costs, then read the report.





8.1 Full example: adding a product and finding its real margin

- The seller opens Categories and adds "Electronics" if it does not exist yet (POST `/categories/add`).
- On Create Product, the category dropdown is filled from GET `/categories`.
- The seller fills in the name, selling price, purchase cost and currency, then clicks Save Product.
- `CreateProduct()` sends POST `/products/add` and, on success, the app routes straight to `/cost-profile/:id` for the new product.
- On the Cost Profile page the seller enters the return rate, gateway fee, packaging, shipping and miscellaneous cost.
- `saveAndCalculateCost()` sends POST `/product/cost-profile/add/:id` — this is where the backend computes the perceived margin, real margin and margin gap.
- The app navigates to the Report page, which calls GET `/product/report/:id` and displays the three stat cards plus the full cost breakdown.
- The same product now also appears on the Dashboard, ranked against every other product by how large its margin gap is.

9. Known Issues and Possible Improvements

A few points worth tracking as the project continues into its next deliverable:

- No central Axios instance: the base URL `http://localhost:3000` is repeated in every view, so moving the API would mean editing each file individually.
- No shared state layer (Pinia/Vuex): data such as the category list is re-fetched by every page that needs it instead of being loaded once.
- `router/index.js` declares two routes named `edit-product` (`/product/edit` and `/product/edit/:id`); route names should be unique.
- `EditProduct.vue` builds its GET request as `http://localhost:3000//product/:id` — the doubled slash should be cleaned up.
- The imported `validateProduct`, `ValidateCategory` and `validateCostProfile` helpers are not yet called before a form submits; today the per-field `@input` handlers do that work instead.

10. Conclusion

This deliverable builds the client side of MarginLens: a Vue 3 single-page application that lets a seller manage products and categories, record the hidden costs behind each product, and see how much of its margin those hidden costs are quietly taking away. Every page is a small, self-contained component that asks the backend API for exactly the data it needs with Axios, which keeps the codebase easy to follow even without a shared store. The next step for the project is connecting this client to a working Express/MongoDB (or similar) backend that implements the ten endpoints listed in Section 7, and tidying up the small inconsistencies noted in Section 9.